
Московский физико-технический институт
Физтех-школа прикладной математики и информатики (ФПМИ)

Отчёт по математическому практикуму

1 курс | 2 семестр 2025/26

Студент: Юрий Гришин

Группа: Б05-506

Преподаватель: Беклемышева Е. А.

Дата: May 16, 2026

Студент: Артём Вотяков

Группа: Б05-506

Преподаватель: Беклемышева Е. А.

Дата: May 16, 2026

Contents

I. Лабораторные работы	4
1 Лабораторная работа №1. Построение сеток в gmsh	4
1.1 Цель работы	4
1.2 Этап 1: Тор с поллой внутренностью	4
1.2.1 Постановка задачи	4
1.2.2 Геометрия и булевы операции	4
1.2.3 Реализация	5
1.2.4 Результат	6
1.3 Этап 2: Сетка в STL-объекте (карандаш)	7
1.3.1 Постановка задачи	7
1.3.2 Проблема: STL описывает только поверхность	7
1.3.3 Алгоритм классификации поверхностей	7
1.3.4 Реализация	7
1.3.5 Результат	8
1.4 Выводы	8
2 Лабораторная работа №2: Paraview	9
2.1 Цель работы	9
2.2 Постановка задачи	9
2.3 Физическая модель	9
2.4 Реализация	9
2.4.1 Загрузка сетки	9
2.4.2 Временной цикл	10
2.4.3 Запись снапшотов в VTK	10
2.5 Параметры симуляции	10
2.6 Визуализация в ParaView	11
2.7 Выводы	12
3 Лабораторная работа №3: FEniCS	13
3.1 Цель работы	13
3.2 Физическая постановка	13
3.2.1 Неоднородность среды	13
3.2.2 Слабая формулировка	13
3.3 Реализация	13
3.4 Результаты	14
3.5 Выводы	14
II. Микропроект	15
1 Микропроект: задача Стефана (таяние снежинки)	15
1.1 Постановка задачи	15
1.2 Математическая модель	15

1.2.1	Физические основы	15
1.2.2	Энтальпийная формулировка	16
1.2.3	Теплопроводность в зоне фазового перехода	16
1.3	Численный метод	17
1.3.1	Явная конечно-разностная схема	17
1.3.2	Условие устойчивости	17
1.3.3	Граничные условия	17
1.3.4	Двойная буферизация	17
1.4	Валидация	17
1.4.1	Одномерная задача Стефана	18
1.5	Постановки задач (сцены)	18
1.5.1	Таяние снежинки	18
1.5.2	Таяние снежинки на руке	19
1.5.3	Намерзание (сосулька)	19
1.6	Реализация	19
1.6.1	Архитектура	19
1.6.2	Оптимизация рендеринга	19
1.6.3	Интерактивное управление	19
1.7	Результаты	20
1.8	Выводы	20
2	Микропроект 2: ультразвуковой импульс в 3D-модели головы	22
2.1	Постановка задачи	22
2.2	Расчётная модель	22
2.2.1	Геометрия и материалы	22
2.2.2	Предобработка сетки	23
2.3	Физическая постановка	23
2.3.1	Линейная акустика	23
2.3.2	Источник	24
2.3.3	Граничные условия	24
2.4	Численный метод	24
2.4.1	Слабая формулировка	24
2.4.2	Полудискретная система	25
2.4.3	Явная схема по времени	25
2.4.4	Условие устойчивости	25
2.5	Реализация	26
2.5.1	Структура проекта	26
2.5.2	Контроль устойчивости	26
2.6	Результаты	26
2.7	Выводы	27
III.	Макропроект	28
1	Макропроект: тепловой блок задачи LSP	28
1.1	Постановка задачи	28
1.2	Физическая постановка	28
1.2.1	Геометрия и масштабы	28
1.2.2	Уравнение энергии	28

1.2.3	Материал	29
1.2.4	Источник лазера	29
1.3	Численный метод	29
1.3.1	Дивергентная схема	29
1.3.2	Устойчивость	30
1.3.3	Граничные условия	30
1.3.4	Линейная термоупругость	30
1.3.5	Критерий пластичности	31
1.4	Реализация	31
1.4.1	Архитектура	31
1.4.2	Экспорт	32
1.5	Результаты	32
1.6	Направления развития	34
1.7	Выводы	34
Список литературы		35

I. Лабораторные работы

1. Лабораторная работа №1. Построение сеток в gmsh

1.1. Цель работы

Метод конечных элементов (МКЭ) сводит дифференциальное уравнение в частных производных к системе алгебраических уравнений, определённых на дискретном разбиении области — *сетке*. Качество этого разбиения напрямую определяет точность и сходимость численного метода: сильно вытянутые или «почти вырожденные» тетраэдры ухудшают обусловленность матрицы жёсткости и порождают паразитные ошибки.

Цель работы — освоить построение таких разбиений с помощью библиотеки **gmsh**: задание геометрии из аналитических примитивов, обработка внешних STL-файлов и контроль качества элементов.

1.2. Этап 1: Тор с полостью внутри

1.2.1. Постановка задачи

Требовалось построить тетраэдральную сетку тора с полостью внутренней полостью: не менее 3–4 тетраэдров должны уместиться в толщину стенки.

Это требование нетривиально: если h слишком велико, стенка покрывается одним-двумя элементами и численный метод не разрешает температурного (или механического) поля внутри неё. Минимальное число элементов на толщину — это по сути дискретный аналог *условия Найквиста*: нужно не менее двух-трёх точек на «полуволну» поля.

1.2.2. Геометрия и булевы операции

Полый тор строится в два шага. Сначала задаётся профиль (окружность в плоскости xOz) четырьмя дугами с центром в точке $(R, 0, 0)$, затем профиль вращается на 2π вокруг оси z :

$$\mathbf{x}(\theta, \phi) = ((R + r \cos \theta) \cos \phi, (R + r \cos \theta) \sin \phi, r \sin \theta), \quad \theta, \phi \in [0, 2\pi). \quad (1)$$

Полость получается булевой операцией **cut**: из тела с параметрами $(R = 4, r = 1)$ вычитается тело $(R = 4, r = 0.5)$. Оставшаяся оболочка имеет толщину стенки $\Delta r = 0.5$. При характеристическом размере элемента $h = 0.2$ в толщину укладывается $\lfloor 0.5/0.2 \rfloor + 1 = 3-4$ тетраэдра — ровно то, что требуется.

1.2.3. Реализация

```
1 int CreateTorusProfile(const TorusParams &params) {
2     double R = params.R, r = params.r;
3     int p1 = gmsh::model::occ::addPoint(R + r, 0.0, 0.0);
4     int p2 = gmsh::model::occ::addPoint(R, 0.0, r);
5     int p3 = gmsh::model::occ::addPoint(R - r, 0.0, 0.0);
6     int p4 = gmsh::model::occ::addPoint(R, 0.0, -r);
7     int pc = gmsh::model::occ::addPoint(R, 0.0, 0.0);
8     int a1 = gmsh::model::occ::addCircleArc(p1, pc, p2);
9     int a2 = gmsh::model::occ::addCircleArc(p2, pc, p3);
10    int a3 = gmsh::model::occ::addCircleArc(p3, pc, p4);
11    int a4 = gmsh::model::occ::addCircleArc(p4, pc, p1);
12    return gmsh::model::occ::addCurveLoop({a1, a2, a3, a4});
13 }
14
15 // rotation around z-axis
16 gmsh::model::occ::revolve(
17     {{2, surfTag}}, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 2.0 * M_PI, out);
```

Listing 1: Профиль и вращение тора

```
1 TorusParams outer{4.0, 1.0}; // outer span
2 TorusParams inner{4.0, 0.5}; // inner span
3 auto outerVol = CreateTorus(outer);
4 auto innerVol = CreateTorus(inner);
5 gmsh::model::occ::synchronize();
6 gmsh::model::occ::cut(cutObjects, toolObjects,
7     outMap, outMapTool, -1, true, true);
```

Listing 2: Вычитаем полость тора

1.2.4. Результат

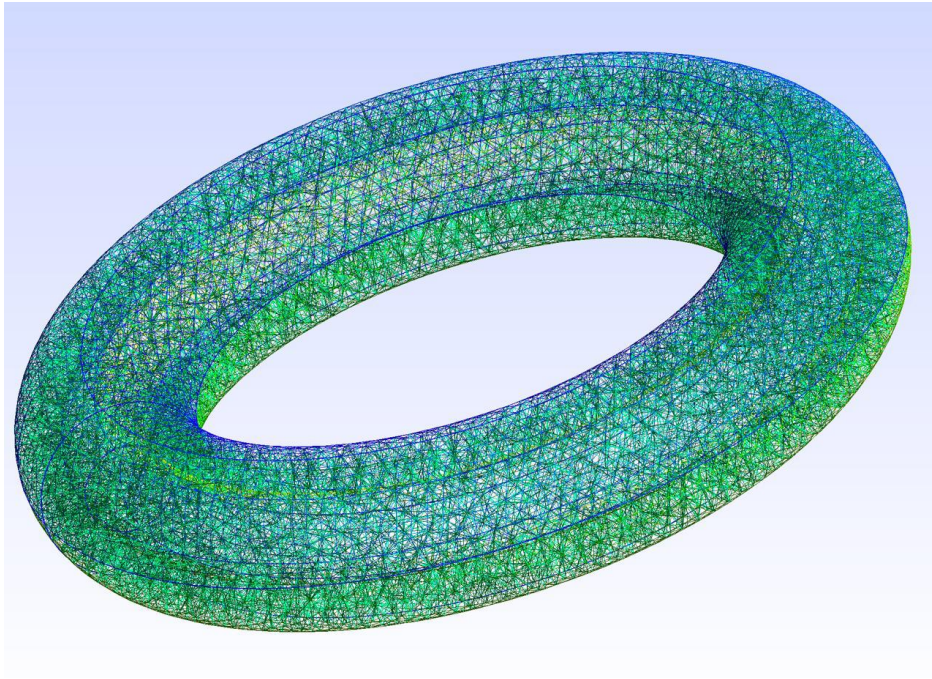


Figure 1: Тетраэдральная сетка полого тора

Вид сбоку показывает, что в толщину стенки укладывается 3–4 элемента, как и планировалось. Внутри полости элементы вытянутые, но это неизбежно при таком соотношении размеров: радиус R в 4 раза больше толщины Δr , и при $h = 0.2$ по окружности укладывается примерно 20 элементов, а по толщине — 3–4.

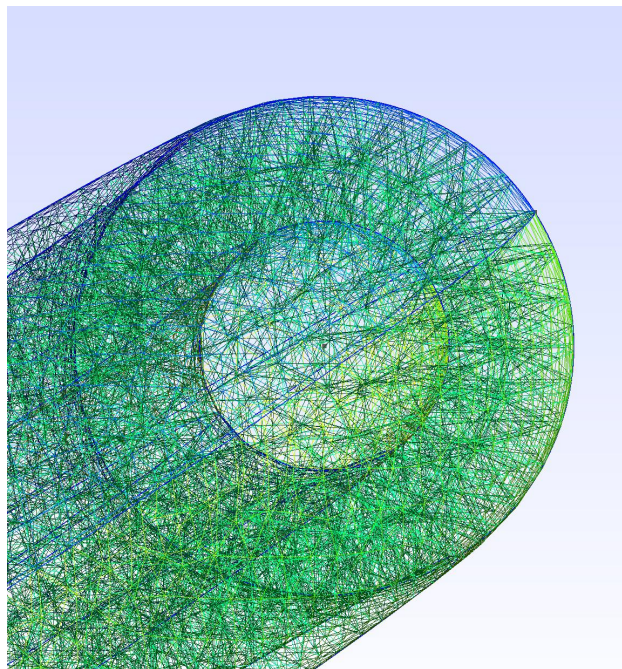


Figure 2: Тетраэдральная сетка полого тора, вид сбоку

1.3. Этап 2: Сетка в STL-объекте (карандаш)

1.3.1. Постановка задачи

Граница реальных инженерных объектов часто задаётся не аналитически, а триангулированной поверхностью — форматом STL. Это типичный вывод систем 3D-моделирования и КТ-сканеров. Задача — построить объёмную тетраэдральную сетку *внутри* такого объекта.

1.3.2. Проблема: STL описывает только поверхность

STL содержит набор треугольных граней, но не несёт информации об объёме и не имеет топологии (нет понятия «рёбра», «вершины» в смысле B-Rep). Чтобы построить объёмную сетку, gmsh должен:

1. Классифицировать грани по геометрическим признакам (выявить острые рёбра — границы патчей);
2. Параметризовать поверхности — задать гладкую геометрию каждого патча, пригодную для проецирования узлов сетки;
3. Сформировать замкнутый объём и заполнить его тетраэдрами.

1.3.3. Алгоритм классификации поверхностей

Команда `classifySurfaces( $\alpha$ )` разделяет триангуляцию на патчи: два соседних треугольника относятся к *одному* патчу, если двугранный угол между ними меньше α . Острые рёбра (угол $\geq \alpha$) становятся границами патчей. Для карандаша $\alpha = 40^\circ$ отделяет грани ствола от граней заточки и торца.

После классификации `createGeometry` строит аналитическое описание каждого патча через B-сплайны — это даёт правильные нормали и возможность точно проецировать узлы на поверхность при генерации сетки.

1.3.4. Реализация

```
1 gmsh::merge("pencil.stl");
2 gmsh::model::mesh::classifySurfaces(
3     40 * M_PI / 180., true, false, 180 * M_PI / 180.);
4 gmsh::model::mesh::createGeometry();
5
6 vector<pair<int,int>> s;
7 gmsh::model::getEntities(s, 2);
8 vector<int> sl;
9 for (auto surf : s) sl.push_back(surf.second);
10
11 int l = gmsh::model::geo::addSurfaceLoop(sl);
12 gmsh::model::geo::addVolume({l});
13 gmsh::model::geo::synchronize();
14
15 gmsh::option::setNumber("Mesh.CharacteristicLengthMin", 2.0);
16 gmsh::option::setNumber("Mesh.CharacteristicLengthMax", 2.0);
17 gmsh::model::mesh::generate(3);
18 gmsh::write("pencil.msh");
```

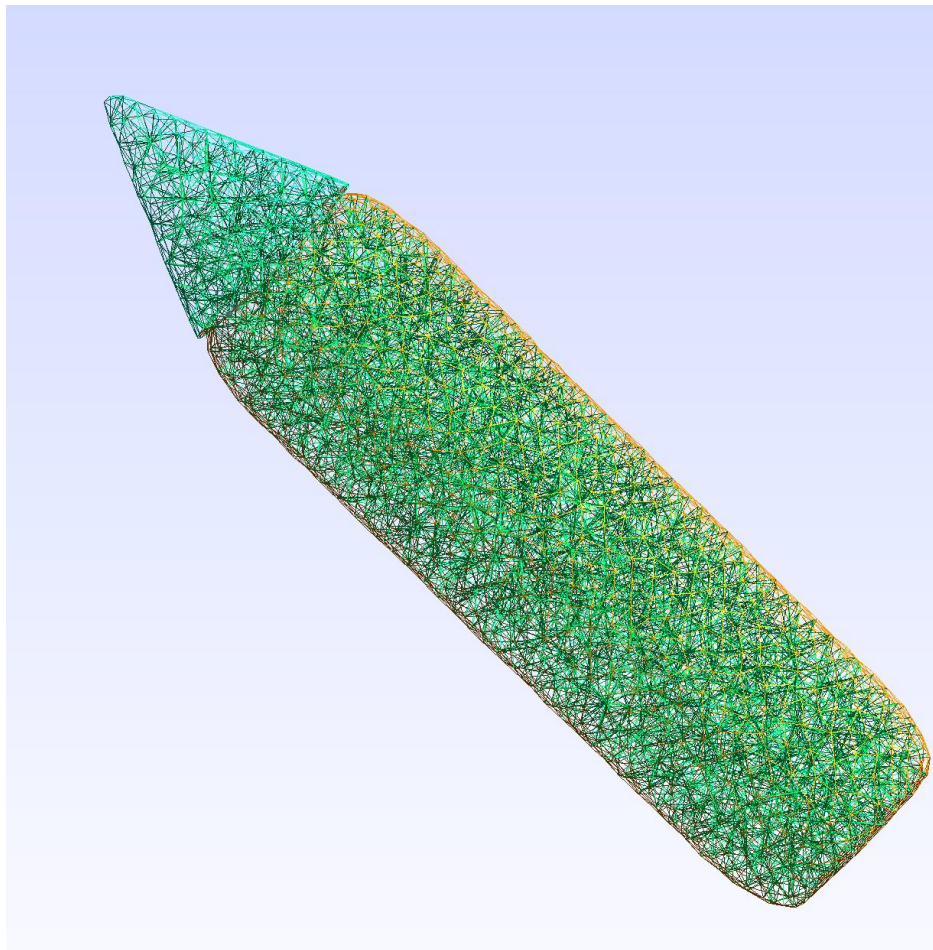
Listing 3: Загрузка STL и построение объёмной сетки**1.3.5. Результат**

Figure 3: Тетраэдральная сетка внутри STL-карандаша, $h = 2.0$.

1.4. Выводы

В работе освоены два принципиально разных подхода к построению тетраэдральных сеток в gmsh.

Конструктивная геометрия (CSG/OCC). Область задаётся аналитически — через примитивы и булевы операции.

Восстановление из STL. Область задаётся дискретной поверхностью без аналитического описания. Но стоит заметить, что иногда может быть очень трудно разбить сложную STL-модель на патчи, пригодные для генерации качественной сетки. В таких случаях нужно как-то вручную трюхать модель.

2. Лабораторная работа №2: Paraview

2.1. Цель работы

Освоить запись результатов расчёта в формат VTK и визуализацию нестационарных процессов в ParaView на примере распространения ударной волны в трёхмерной неструктурированной сетке.

2.2. Постановка задачи

На сетке из предыдущей лабы (карандаш `pencil.stl`) моделируется распространение сферической ударной волны от точечного источника. В каждом узле сетки на каждом временном шаге вычисляются скалярное поле давления p и векторное поле скоростей $\mathbf{v}$.

2.3. Физическая модель

Волна задаётся аналитически как гауссов импульс, распространяющийся со скоростью c от точки (x_0, y_0, z_0) :

$$p(\mathbf{x}, t) = v_0 \exp\left(-\frac{(r - ct)^2}{\sigma^2}\right) \cdot A(r), \quad r = |\mathbf{x} - \mathbf{x}_0|, \quad (2)$$

где затухание $A(r)$ моделирует геометрическое и вязкое рассеяние:

$$A(r) = \frac{e^{-\alpha r}}{1 + \beta r}, \quad \alpha = 0.005, \quad \beta = 0.02. \quad (3)$$

Отражения от торцов по оси z учитываются зеркальными источниками с коэффициентом отражения 0.8 (первый порядок) и 0.64 (второй порядок).

2.4. Реализация

2.4.1. Загрузка сетки

Сетка загружается из `.msh`-файла через `gmsh` API, узлы и 3D-элементы сохраняются во внутренних структурах `Node` и `Cell`.

```

1 gmsh::model::mesh::getNodes(nodeTags, coords, parametricCoords);
2 for (size_t i = 0; i < nodeTags.size(); ++i) {
3     nodes_[i].x = coords[3*i + 0];
4     nodes_[i].y = coords[3*i + 1];
5     nodes_[i].z = coords[3*i + 2];
6     tag_to_index[nodeTags[i]] = i;
7 }
```

Listing 4: Загрузка узлов из `.msh`

2.4.2. Временной цикл

На каждом шаге для каждого узла вычисляется вклад прямой волны и четырёх зеркальных источников:

```

1 void PointHit(Node &node, double t) {
2     applyWave(hit_center_, node, t, c_, v0_, sigma_);
3
4     Node mirror_top(cx, cy, 2.0*z_max_ - hit_center_.z);
5     applyWave(mirror_top, node, t, c_, v0_*0.8, sigma_);
6
7     Node mirror_bot(cx, cy, 2.0*z_min_ - hit_center_.z);
8     applyWave(mirror_bot, node, t, c_, v0_*0.8, sigma_);
9 }

```

Listing 5: Вычисление поля в узле

2.4.3. Запись снимков в VTK

Каждый кадр записывается в отдельный .vtu-файл:

```

1 auto grid = vtkSmartPointer<vtkUnstructuredGrid>::New();
2 auto smth = vtkSmartPointer<vtkDoubleArray>::New();
3 auto vel = vtkSmartPointer<vtkDoubleArray>::New();
4 vel->SetNumberOfComponents(3);
5
6 for (size_t i = 0; i < nodes_.size(); i++) {
7     dumpPoints->InsertNextPoint(nodes_[i].x, nodes_[i].y, nodes_[i].z);
8     double v[3] = {nodes_[i].vx, nodes_[i].vy, nodes_[i].vz};
9     vel->InsertNextTuple(v);
10    smth->InsertNextValue(nodes_[i].smth);
11 }
12 string fileName = "pencil-step-" + to_string(snap_number) + ".vtu";
13 writer->SetFileName(fileName.c_str());
14 writer->Write();

```

Listing 6: Запись кадра в VTK

2.5. Параметры симуляции

Table 1: Параметры расчёта

Параметр	Описание	Значение
c	Скорость волны	15 ед./с
v_0	Амплитуда	15.0
σ	Ширина гауссова импульса	3.0
Δt	Шаг по времени	0.05 с
T	Длительность симуляции	20.0 с
(x_0, y_0, z_0)	Источник	(2, 0, 85)

2.6. Визуализация в ParaView

Используя ParaView, была создана анимация, показывающая распространение ударной волны внутри карандаша.

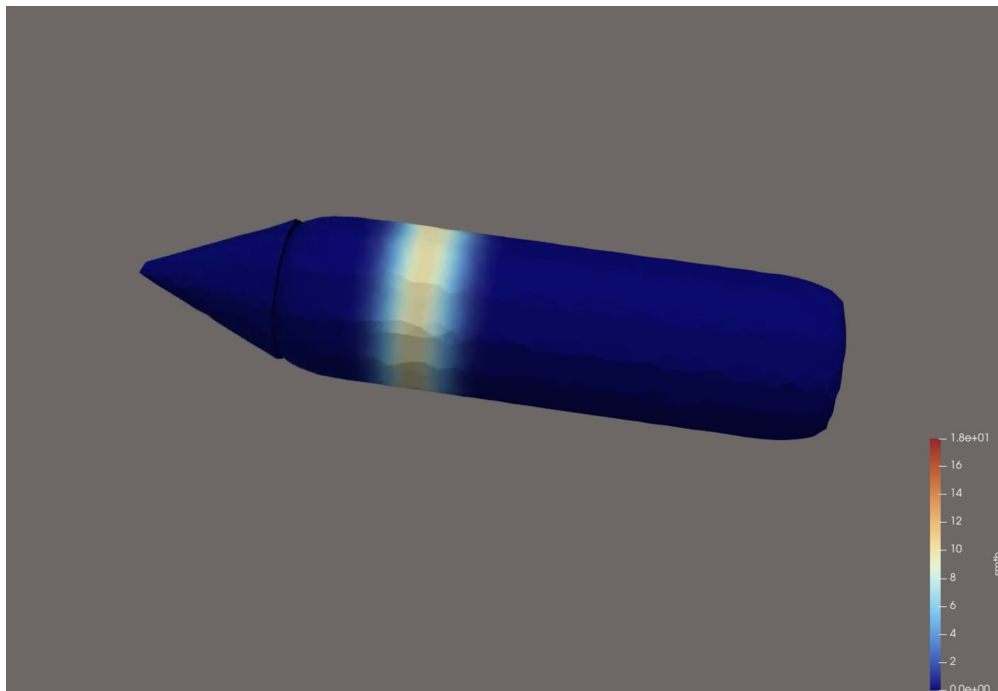


Figure 4: Кадр из анимации, показывающий распространение ударной волны.

См. [Видео на YouTube](#).

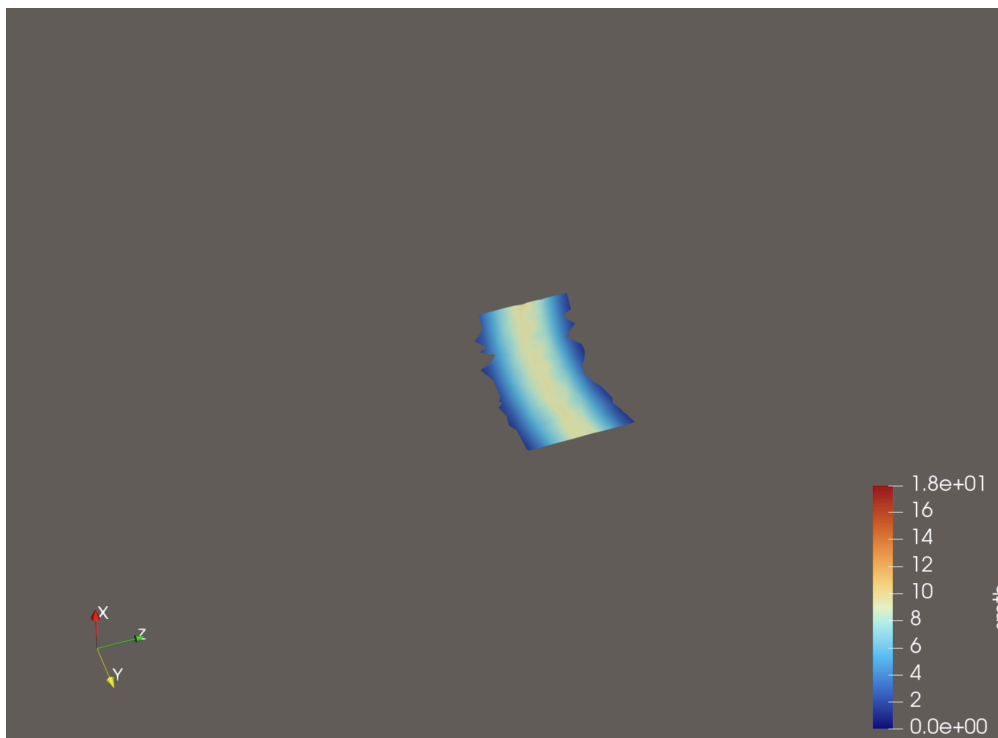


Figure 5: Сечение вдоль оси карандаша, показывающее распространение ударной волны.

2.7. Выводы

Реализован полный цикл расчёта и визуализации нестационарного процесса: загрузка STL-сетки через gmsh, аналитический расчёт поля волны с учётом отражений, запись серии .vtu-снимков и визуализация в ParaView. Использование неструктурированной сетки позволило корректно передать геометрию объекта произвольной формы.

3. Лабораторная работа №3: FEniCS

3.1. Цель работы

Научиться ставить эллиптические краевые задачи в FEniCS и решать их МКЭ. Физический полигон — электрический диполь в плоской проводящей среде с локальной неоднородностью.

3.2. Физическая постановка

Стационарный потенциал ϕ в среде с переменной проводимостью $\lambda(\mathbf{x})$ подчиняется уравнению:

$$-\nabla \cdot (\lambda \nabla \phi) = 0, \quad \mathbf{x} \in [-1, 1]^2. \quad (4)$$

Электроды — малые окружности радиуса 0.1 с фиксированным потенциалом:

$$\phi|_{\mathbf{x}_+} = +1, \quad \phi|_{\mathbf{x}_-} = -1, \quad \mathbf{x}_{\pm} = (\pm 0.5, 0). \quad (5)$$

3.2.1. Неоднородность среды

В центре области сидит слабопроводящее гауссово включение:

$$\lambda(\mathbf{x}) = \frac{1}{100 e^{-25|\mathbf{x}|^2} + 1}. \quad (6)$$

При $|\mathbf{x}| \rightarrow 0$ имеем $\lambda \approx 0.01$, вдали — $\lambda \rightarrow 1$. Ширина включения — $\sigma = 1/\sqrt{25} = 0.2$.

3.2.2. Слабая формулировка

Интегрируя по частям, получаем вариационную форму:

$$\int_{\Omega} \lambda \nabla \phi \cdot \nabla v \, d\mathbf{x} = 0 \quad \forall v \in V_0. \quad (7)$$

Неоднородность $\lambda(\mathbf{x})$ входит непосредственно в билинейную форму — дополнительных модификаций FEniCS не требует.

3.3. Реализация

Основная часть кода была взята из [репозитория](#) и адаптирована под текущую задачу.

```

1 mesh = RectangleMesh(Point(-1, -1), Point(1, 1), 128, 128)
2 V     = FunctionSpace(mesh, "Lagrange", 1)
3
4 bc_plus  = DirichletBC(V, Constant( 1.0),
5     lambda x, _: (x[0]-0.5)**2 + x[1]**2 < 0.01, "pointwise")
6 bc_minus = DirichletBC(V, Constant(-1.0),
7     lambda x, _: (x[0]+0.5)**2 + x[1]**2 < 0.01, "pointwise")
8
9 la = Expression("1/(100*exp(-25*(x[0]*x[0]+x[1]*x[1]))+1)", degree=2)
10

```

```

11 u, v = Function(V), TestFunction(V)
12 solve(inner(grad(u), grad(v)) * la * dx == 0, u, [bc_plus, bc_minus])

```

Listing 7: Сетка, условия Дирихле и решение

```

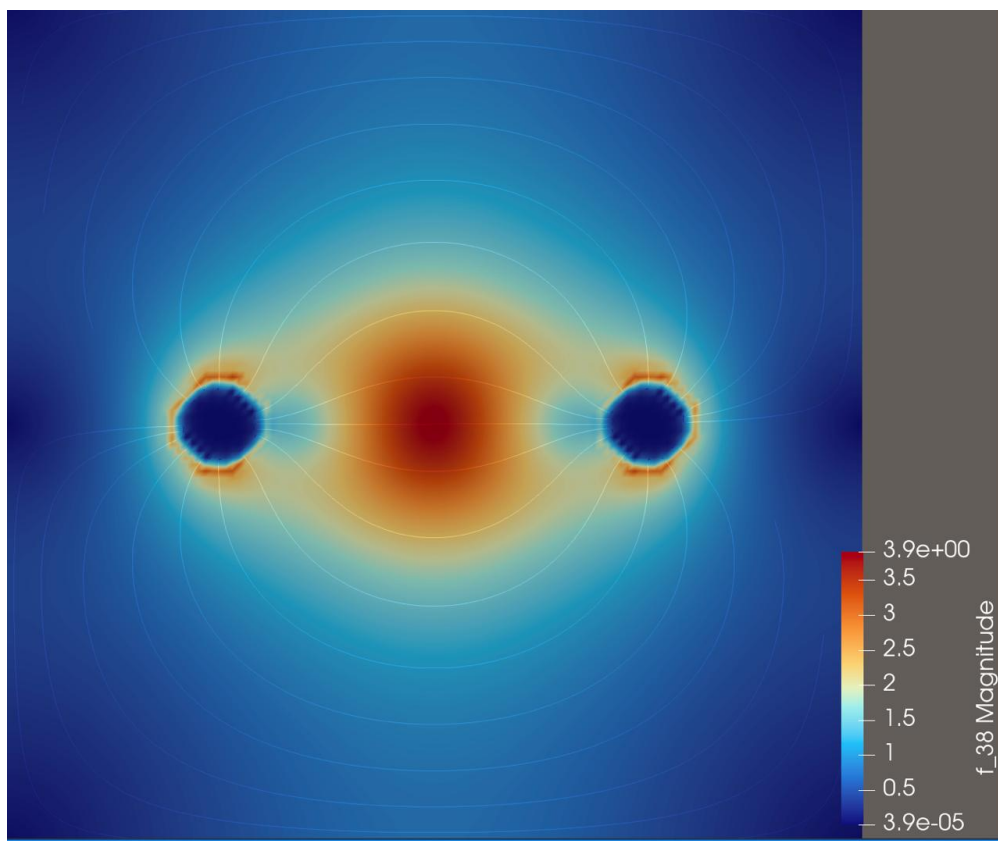
1 W = VectorFunctionSpace(mesh, "Lagrange", 1)
2 E = project(-grad(u), W)
3 j = project(-la * grad(u), W)

```

Listing 8: Поля E и j через проекцию

Результаты сохранялись в `.pvd/.vtu` для ParaView.

3.4. Результаты



Вблизи электродов поле имеет ожидаемую дипольную структуру. Линии тока j обтекают слабопроводящее включение — оно действует как препятствие, перенаправляя ток в обход центра.

3.5. Выводы

Реализовано МКЭ-решение задачи диполя в неоднородной среде. Поведение поля качественно верное: ток обтекает высокоомное включение.

II. Микропроект

1. Микропроект: задача Стефана (таяние снежинки)

1.1. Постановка задачи

Микропроект посвящён двумерному моделированию фазового перехода лёд–вода (задача Стефана) энтальпийным методом конечных разностей. Задача Стефана — это задача теплопроводности с подвижной границей: при плавлении или замерзании граница раздела фаз перемещается, поглощая или выделяя скрытую теплоту L .

Классическая формулировка требует отслеживания фронта фазового перехода и выполнения на нём условия Стефана:

$$\rho L \frac{ds}{dt} = k_l \left. \frac{\partial T_l}{\partial n} \right|_s - k_s \left. \frac{\partial T_s}{\partial n} \right|_s, \quad (8)$$

где $s(t)$ — положение фронта, k_l, k_s — теплопроводности жидкой и твёрдой фаз, $\partial T / \partial n$ — производные температуры по нормали к границе с каждой стороны. Это условие выражает баланс тепловых потоков: разница потоков идёт на плавление (или замерзание) вещества.

В двумерном случае отслеживание фронта становится технически сложной задачей. Энтальпийный метод позволяет обойти эту проблему, заменяя явное условие на границе неявной обработкой через функцию $T(H)$.

1.2. Математическая модель

1.2.1. Физические основы

Рассмотрим среду, состоящую из одного вещества (вода/лёд) с фазовым переходом I рода при температуре T_{melt} . При фазовом переходе I рода выделяется или поглощается скрытая теплота L (Дж/кг) — энергия, необходимая для разрыва межмолекулярных связей кристаллической решётки льда, не изменяющая при этом температуру вещества.

Теплоперенос в каждой фазе описывается уравнением Фурье:

$$\rho c \frac{\partial T}{\partial t} = k \nabla^2 T, \quad (9)$$

где ρ — плотность, c — удельная теплоёмкость, k — коэффициент теплопроводности. Это уравнение следует из закона сохранения энергии и закона Фурье $\mathbf{q} = -k \nabla T$, связывающего тепловой поток с градиентом температуры.

Параметры вещества различаются для твёрдой и жидкой фаз:

Параметр	Лёд	Вода
Теплопроводность k , Вт/(м·К)	2.2	0.6
Удельная теплоёмкость c , Дж/(кг·К)	2090	4180
Скрытая теплота L , Дж/кг	334 000	
Плотность ρ , кг/м ³	1000	

Температуропроводность $\alpha = k/(\rho c)$ определяет скорость диффузии тепла: для льда $\alpha_s \approx 1,05 \cdot 10^{-6}$ м²/с, для воды $\alpha_l \approx 1,44 \cdot 10^{-7}$ м²/с. Лёд проводит тепло почти на порядок быстрее воды.

1.2.2. Энтальпийная формулировка

Введём объёмную удельную энтальпию H (Дж/м³), отсчитанную от точки плавления. Энтальпия — это термодинамический потенциал, равный полной тепловой энергии, накопленной единицей объёма вещества. Уравнение теплопроводности в терминах энтальпии принимает единую форму для обеих фаз:

$$\frac{\partial H}{\partial t} = k \nabla^2 T. \quad (10)$$

Связь температуры и энтальпии кусочно-линейна:

$$T(H) = \begin{cases} T_{\text{melt}} + \frac{H}{\rho c_s}, & H < 0 \quad (\text{лёд}), \\ T_{\text{melt}}, & 0 \leq H \leq \rho L \quad (\text{фазовый переход}), \\ T_{\text{melt}} + \frac{H - \rho L}{\rho c_l}, & H > \rho L \quad (\text{вода}). \end{cases} \quad (11)$$

Физический смысл трёх областей:

- $H < 0$: вещество полностью в твёрдой фазе; вся энергия расходуется на изменение температуры;
- $0 \leq H \leq \rho L$: вещество находится в процессе фазового перехода; подводимая энергия расходуется на разрушение кристаллической решётки (скрытая теплота), температура при этом остаётся равной T_{melt} ;
- $H > \rho L$: вещество полностью в жидкой фазе; вся скрытая теплота уже поглощена, дальнейшая энергия снова идёт на нагрев.

Ключевое достоинство энтальпийного метода: граница раздела фаз обрабатывается автоматически. Ячейки, в которых $0 \leq H \leq \rho L$, находятся «в процессе» фазового перехода — фронт плавления «размазывается» по нескольким ячейкам сетки, не требуя явного отслеживания его положения.

1.2.3. Теплопроводность в зоне фазового перехода

В ячейках, находящихся в процессе плавления ($0 \leq H \leq \rho L$), теплопроводность интерполируется между фазами:

$$k(H) = (1 - f) k_s + f k_l, \quad f = \frac{H}{\rho L}, \quad (12)$$

где $f \in [0, 1]$ — доля жидкой фазы. Это обеспечивает плавный переход теплофизических свойств через фронт.

1.3. Численный метод

1.3.1. Явная конечно-разностная схема

Уравнение (10) дискретизируется на равномерной квадратной сетке $N \times N$ с шагом $h = D/N$, где D — размер области. Лапласиан аппроксимируется стандартным пятиточечным шаблоном:

$$H_{i,j}^{n+1} = H_{i,j}^n + \Delta t \cdot k_{i,j} \cdot \frac{T_{i+1,j}^n + T_{i-1,j}^n + T_{i,j+1}^n + T_{i,j-1}^n - 4T_{i,j}^n}{h^2}, \quad (13)$$

где $k_{i,j} = k(H_{i,j}^n)$ — теплопроводность, зависящая от фазы.

Схема является **явной**: значения на новом временном слое $n+1$ вычисляются непосредственно из значений на слое n , без решения системы линейных уравнений. Это даёт простую реализацию, но накладывает ограничение на шаг по времени.

1.3.2. Условие устойчивости

Явная схема устойчива при выполнении условия CFL:

$$\Delta t \leq \frac{h^2 \rho c_{\min}}{4 k_{\max}}, \quad (14)$$

где множитель 4 соответствует двумерному случаю (четыре соседа в шаблоне), $c_{\min}$ и $k_{\max}$ — наименьшая теплоёмкость и наибольшая теплопроводность среди фаз. Физический смысл: за один шаг по времени тепло не должно «перескочить» через ячейку.

Для используемых параметров ($h = 10^{-4}$ м, $k_{\max} = 2,2$, $c_{\min} = 2090$, $\rho = 1000$):

$$\Delta t_{\max} \approx 2,4 \cdot 10^{-3} \text{ с.}$$

1.3.3. Граничные условия

Реализованы два типа граничных условий:

Фиксированные (Дирихле): температура на границе не меняется — $T|_{\partial\Omega} = T_0$. Используется для моделирования контакта с телом заданной температуры (например, рука при 36°C).

Адиабатические (Нейман): тепловой поток через границу равен нулю — $\partial T / \partial n|_{\partial\Omega} = 0$. Реализуется копированием энтальпии из ближайшей внутренней ячейки. Используется для замкнутых систем без теплообмена с окружением.

1.3.4. Двойная буферизация

При обновлении сетки по схеме (13) значения $T_{i\pm 1,j}^n$ и $T_{i,j\pm 1}^n$ должны быть со *старого* временного слоя. Поэтому используется буфер H_{new} : сначала все новые значения записываются в буфер, затем буфер копируется обратно в основной массив.

1.4. Валидация

1.4.1. Одномерная задача Стефана

Для верификации численной схемы рассмотрена одномерная задача: полупространство льда при $T = T_{\text{melt}}$, левая граница мгновенно нагревается до T_{hot} . Аналитическое решение даёт положение фронта плавления:

$$s(t) = 2\lambda\sqrt{\alpha_l t}, \quad (15)$$

где $\alpha_l = k_l/(\rho c_l)$ — температуропроводность жидкой фазы, а λ — корень трансцендентного уравнения:

$$\text{St} = \lambda\sqrt{\pi} e^{\lambda^2} \text{erf}(\lambda), \quad (16)$$

где $\text{St} = c_l(T_{\text{hot}} - T_{\text{melt}})/L$ — число Стефана, характеризующее отношение теплоты, затраченной на нагрев жидкости, к скрытой теплоте плавления. При $T_{\text{hot}} = 50^\circ\text{C}$:

$$\text{St} = \frac{4180 \cdot 50}{334\,000} \approx 0,626.$$

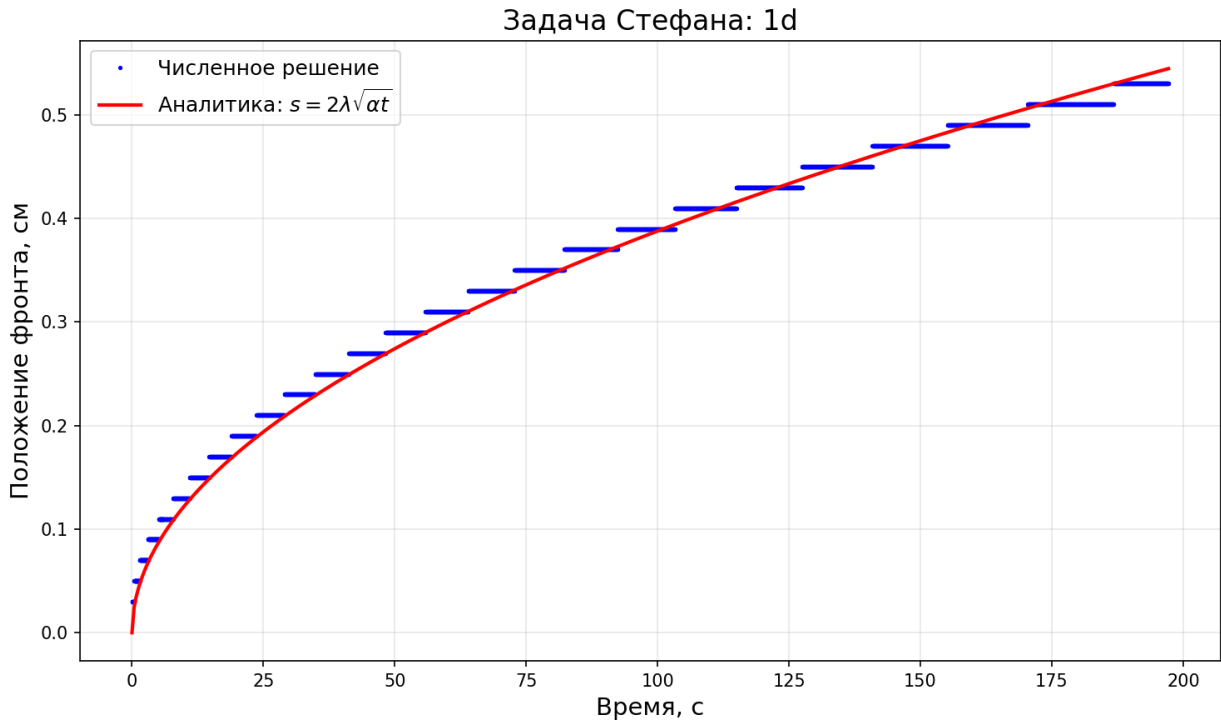


Figure 6: Сравнение численного положения фронта $s(t)$ с аналитическим решением. Численный фронт воспроизводит закон $s \propto \sqrt{t}$.

1.5. Постановки задач (сцены)

1.5.1. Таяние снежинки

Снежинка задаётся PNG-маской: тёмные пиксели маски инициализируются как лёд ($H = \rho c_s \cdot (T_0 - T_{\text{melt}}) < 0$), окружающая среда — как тёплая вода ($H = \rho L + \rho c_l \cdot (T_w - T_{\text{melt}}) > \rho L$). Тонкие лучи тают первыми, так как у них наибольшее отношение поверхности к объёму: тепло проникает к центру тонкого элемента быстрее.

1.5.2. Таяние снежинки на руке

Снежинка помещается на нижнюю границу области с фиксированной температурой $T_{\text{hand}} = 36^\circ\text{C}$ (температура тела человека). Окружающий воздух моделируется как лёд при $T \approx 0^\circ\text{C}$. При размере снежинки ~ 5 мм и реальных теплофизических параметрах время полного таяния составляет 2–3 с, что качественно совпадает с повседневным опытом.

1.5.3. Намерзание (сосулька)

Вода при $+2^\circ\text{C}$, верхняя стенка охлаждена до -20°C , боковые стенки при $+5^\circ\text{C}$. Лёд нарастает сверху вниз; тёплые боковые границы не дают ему расширяться, формируя вытянутую структуру. Эта постановка демонстрирует обратимость фазового перехода: тот же солвер без изменений моделирует и плавление, и замерзание.

1.6. Реализация

1.6.1. Архитектура

Проект реализован на C++17 с разделением на модули:

- **core/** — физика (Grid, Solver, Material). Не зависит от библиотеки визуализации;
- **render/** — визуализация (Renderer, Colormap). Зависит от raylib, не знает о деталях физики;
- **scenes/** — начальные условия. Каждая сцена реализует интерфейс Scene;
- **export/** — вывод данных в CSV для анализа в Python.

Зависимости однонаправлены: **render** $\rightarrow$ **core** (только чтение), **scenes** $\rightarrow$ **core**. Циклических зависимостей нет.

1.6.2. Оптимизация рендеринга

Наивная отрисовка $N \times N$ прямоугольников через **DrawRectangle** ($\sim 40\,000$ вызовов при $N = 200$) даёт ~ 4 FPS. Вместо этого температурное поле записывается в массив пикселей, загружается как текстура на GPU (**UpdateTexture**) и отрисовывается одним вызовом **DrawTextureEx**. Результат: ~ 60 FPS.

1.6.3. Интерактивное управление

Raylib-приложение поддерживает:

- [1] - [5] — переключение сцен;
- [SPACE] — пауза / продолжение;
- [R] — сброс состояния;
- [LMB] — локальный нагрев мышью;
- [RMB] — локальное охлаждение мышью;
- [F] — запись кадров в PNG.

1.7. Результаты



Figure 7: Таяние снежинки. Синий — лёд, белый — фазовый переход, красный — вода. Тонкие лучи исчезают первыми.

См. [Видео на YouTube](#).

Качественные наблюдения:

1. Тонкие элементы снежинки тают быстрее массивных — следствие большего отношения площади поверхности к объёму.
2. Фронт плавления в 1D воспроизводит аналитический закон $s \propto \sqrt{t}$.
3. Намерзание и плавление обрабатываются одним и тем же кодом без модификаций — обратимость фазового перехода I рода.

1.8. Выводы

Реализован двумерный решатель задачи Стефана энтальпийным методом на явной конечно-разностной схеме. Энтальпийная формулировка позволяет избежать явного отслеживания фронта плавления, сохраняя физически корректную картину фазового

перехода. Валидация на одномерном аналитическом решении подтвердила корректность реализации.

Продемонстрированы три режима: таяние (снежинка в тёплой воде), контактное плавление (снежинка на руке) и намерзание (рост льда от холодного источника). Все режимы обрабатываются единым солвером без модификации уравнений — это прямое следствие энтальпийной формулировки.

2. Микропроект 2: ультразвуковой импульс в 3D-модели головы

2.1. Постановка задачи

Второй микропроект посвящён моделированию распространения короткого ультразвукового импульса в трёхмерной анатомической модели головы. Искомая величина — акустическое давление $p(\mathbf{x}, t)$, распространяющееся по неоднородной среде: мягким тканям, костям, кровеносным сосудам и другим областям, заданным на тетраэдральной сетке.

Цель работы — собрать полный расчётный пайплайн:

1. взять готовую сегментированную 3D-модель головы;
2. перенести материал каждой ячейки в DOLFINx;
3. задать импульсный ультразвуковой источник;
4. решить нестационарную акустическую задачу методом конечных элементов;
5. сохранить результаты в формате `.vtu` для ParaView.

Геометрическая модель взята из репозитория [gcm-3d](#), ветка с моделями `ani3d`. Это готовая тетраэдральная сетка анатомической области с метками материалов. Расчётный код был написан с опорой на готовые примеры из [fenicsx-fus](#), где уже используется DOLFINx для задач ультразвука.

2.2. Расчётная модель

2.2.1. Геометрия и материалы

В каталоге `mini_project/3d-models/` используются два основных VTU-файла:

- `mesh.vtu` — исходная неструктурированная тетраэдральная сетка головы;
- `head_with_materials.vtu` — сетка после предобработки, где к каждой ячейке добавлены поля `material_id`, `rho`, `lambda`, `mu`, `sound_speed`.

В итоговой сетке:

$$N_{\text{points}} = 125\,215, \quad N_{\text{cells}} = 729\,468.$$

Координаты модели используются в миллиметрах, время — в секундах, поэтому скорость звука в коде задаётся в мм/с. Плотность и упругие константы берутся из таблицы материалов, приведенной в том же источнике, что и модель.

Область	Material ID	ρ , г/мм ³	c , мм/с	(λ, μ)
Жир	3, 31	$1,158 \cdot 10^{-3}$	$1,45 \cdot 10^6$	$(3,07 \cdot 10^6, 2,05 \cdot 10^6)$
Мышцы / мягкие ткани	4, 11, 12, 27, 30, 32	$1,00 \cdot 10^{-3}$	$1,58 \cdot 10^6$	$(3,07 \cdot 10^6, 2,05 \cdot 10^6)$
Кость	21	$1,00 \cdot 10^{-3}$	$3,50 \cdot 10^6$	$(7,86 \cdot 10^8, 1,18 \cdot 10^9)$
Кровь	22, 28, 29	$1,00 \cdot 10^{-3}$	$1,57 \cdot 10^6$	$(2,39 \cdot 10^5, 5,00 \cdot 10^2)$
Трахея	24	$2,00 \cdot 10^{-3}$	$1,54 \cdot 10^6$	$(1,43 \cdot 10^7, 3,57 \cdot 10^6)$
Аорта	25	$1,00 \cdot 10^{-3}$	$1,57 \cdot 10^6$	$(9,21 \cdot 10^6, 1,90 \cdot 10^5)$
Вены	26	$1,00 \cdot 10^{-3}$	$1,57 \cdot 10^6$	$(3,289 \cdot 10^7, 6,70 \cdot 10^5)$

В акустической постановке непосредственно используются $\rho(\mathbf{x})$ и $c(\mathbf{x})$. Константы Ламе $\lambda(\mathbf{x})$, $\mu(\mathbf{x})$ сохранены в модели для следующего шага — линейно-упругого расчёта смещений.

2.2.2. Предобработка сетки

Файл `preprocess.py` читает VTU через `meshio`, находит тетраэдральный блок и восстанавливает `Material ID` для каждой ячейки.

Материалы записываются в ячейки:

$$\rho_h, c_h, \lambda_h, \mu_h \in DG_0(\Omega), \quad (17)$$

то есть каждое значение постоянно внутри одного тетраэдра.

2.3. Физическая постановка

2.3.1. Линейная акустика

Ультразвуковой импульс рассматривается как малая добавка давления к равновесному состоянию среды. Поэтому используется линейная акустика: амплитуда возмущения мала по сравнению с модулем объёмного сжатия, а нелинейные эффекты и поглощение в первом приближении не учитываются.

Пусть $\mathbf{v}(\mathbf{x}, t)$ — акустическая скорость частиц среды, $p(\mathbf{x}, t)$ — акустическое давление, $\rho(\mathbf{x})$ — плотность, $c(\mathbf{x})$ — скорость звука. Для малых возмущений:

$$\rho(\mathbf{x}) \frac{\partial \mathbf{v}}{\partial t} = -\nabla p, \quad (18)$$

$$\frac{1}{\rho(\mathbf{x})c^2(\mathbf{x})} \frac{\partial p}{\partial t} + \nabla \cdot \mathbf{v} = 0. \quad (19)$$

Первое уравнение — линеаризованный закон Ньютона: градиент давления разгоняет частицы среды. Второе — связь сжимаемости и давления: при сжатии объёма давление растёт; коэффициент $K = \rho c^2$ играет роль объёмного модуля.

Исключая $\mathbf{v}$, получаем волновое уравнение в неоднородной среде:

$$\frac{1}{\rho(\mathbf{x})c^2(\mathbf{x})} \frac{\partial^2 p}{\partial t^2} - \nabla \cdot \left(\frac{1}{\rho(\mathbf{x})} \nabla p \right) = s(\mathbf{x}, t). \quad (20)$$

Если ρ и c постоянны, оно сводится к обычному уравнению $p_{tt} - c^2 \Delta p = \rho c^2 s$. В данной задаче коэффициенты кусочно-постоянны по тканям, поэтому фронт волны меняет скорость, отражается и преломляется на границах материалов.

2.3.2. Источник

Источник разделён на пространственную и временную части:

$$s(\mathbf{x}, t) = g(\mathbf{x}) q(t). \quad (21)$$

Пространственный профиль — гауссово пятно:

$$g(\mathbf{x}) = A \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_s\|^2}{2\sigma^2}\right). \quad (22)$$

В коде центр $\mathbf{x}_s$ ставится около нижней границы расчётной области:

$$x_s = \frac{x_{\min} + x_{\max}}{2}, \quad y_s = y_{\min} + 0,4(y_{\max} - y_{\min}), \quad z_s = z_{\min}, \quad (23)$$

а ширина берётся как $\sigma = 0,1 D$, где D — максимальный размер bounding box модели.

Временная форма — оконный ультразвуковой burst:

$$q(t) = \begin{cases} \sin(2\pi f \tau) \sin^2\left(\frac{\pi \tau}{T}\right), & 0 \leq \tau \leq T, \\ 0, & \text{иначе,} \end{cases} \quad (24)$$

где

$$\tau = t - t_{\text{start}}, \quad t_{\text{start}} = t_0 - \frac{T}{2}, \quad T = \frac{N}{f}.$$

В расчёте:

$$f = 1 \text{ МГц}, \quad N = 5, \quad t_0 = \frac{5}{f}, \quad T = \frac{5}{f}.$$

Множитель $\sin^2(\pi \tau / T)$ плавно поднимает амплитуду от нуля и плавно возвращает её в ноль. Это уменьшает артефакты, которые появились бы при резком обрыве синуса.

2.3.3. Граничные условия

Явно заданных граничных условий Дирихле в акустическом коде нет. После интегрирования по частям естественное граничное условие имеет вид

$$\frac{1}{\rho} \nabla p \cdot \mathbf{n} \Big|_{\partial\Omega} = 0. \quad (25)$$

Физически это отражающая граница.

2.4. Численный метод

2.4.1. Слабая формулировка

Умножим уравнение (20) на тестовую функцию $v \in V$ и проинтегрируем по области. С учётом естественного граничного условия получаем:

$$\int_{\Omega} \frac{1}{\rho c^2} \frac{\partial^2 p}{\partial t^2} v \, d\mathbf{x} + \int_{\Omega} \frac{1}{\rho} \nabla p \cdot \nabla v \, d\mathbf{x} = q(t) \int_{\Omega} g(\mathbf{x}) v \, d\mathbf{x}. \quad (26)$$

Используется пространство конечных элементов Лагранжа первой степени на тетраэдральной сетке:

$$V_h = \{v_h : v_h|_K \in P_1(K)\}.$$

2.4.2. Полудискретная система

После FEM-дискретизации получаем систему ОДУ:

$$\mathbf{M}\ddot{\mathbf{p}} + \mathbf{K}\mathbf{p} = q(t)\mathbf{F}, \quad (27)$$

где

$$M_{ij} = \int_{\Omega} \frac{1}{\rho c^2} \phi_i \phi_j d\mathbf{x}, \quad (28)$$

$$K_{ij} = \int_{\Omega} \frac{1}{\rho} \nabla \phi_j \cdot \nabla \phi_i d\mathbf{x}, \quad (29)$$

$$F_i = \int_{\Omega} g(\mathbf{x}) \phi_i d\mathbf{x}. \quad (30)$$

Матрица жёсткости $\mathbf{K}$ собирается обычным образом через `assemble_matrix`. Матрица масс используется в `lumped`-виде: вместо полной матрицы хранится диагональный вектор сумм по строкам. Это делает явный шаг дешёвым: не нужно решать линейную систему на каждом временном слое.

2.4.3. Явная схема по времени

Для второй производной по времени используется центральная разность:

$$\ddot{\mathbf{p}}^n \approx \frac{\mathbf{p}^{n+1} - 2\mathbf{p}^n + \mathbf{p}^{n-1}}{\Delta t^2}. \quad (31)$$

Подставляя это в (27), получаем явную схему:

$$\mathbf{p}^{n+1} = 2\mathbf{p}^n - \mathbf{p}^{n-1} + \Delta t^2 \mathbf{M}_L^{-1} (q(t_n)\mathbf{F} - \mathbf{K}\mathbf{p}^n). \quad (32)$$

Начальные условия нулевые:

$$p(\mathbf{x}, 0) = 0, \quad \frac{\partial p}{\partial t}(\mathbf{x}, 0) = 0.$$

2.4.4. Условие устойчивости

Явная схема для волнового уравнения требует шага, согласованного со скоростью распространения волны и минимальным размером элемента. В коде шаг оценивается автоматически:

$$\Delta t = \text{safety} \frac{h_{\min}}{c_{\max}}, \quad \text{safety} = 0,25. \quad (33)$$

Здесь $h_{\min}$ вычисляется через `cpp.mesh.h`.

$$c_{\max} = 3,5 \cdot 10^6 \text{ мм/с}.$$

2.5. Реализация

2.5.1. Структура проекта

Основные файлы:

- `mini_project/src/acoustic_impulse.py` — основной акустический расчёт;
- `mini_project/src/tools/preprocess.py` — загрузка VTU, перенос Material ID, создание DG0-полей;
- `mini_project/src/tools/convert_to_vtu_with_materials.py` — добавление физических параметров в VTU-файл;

Результаты пишутся в папку `mini_project/src/acoustic_vtu_frames_other_source/` как последовательность `.vtu`-кадров.

2.5.2. Контроль устойчивости

Если шаг времени или амплитуда источника выбраны неправильно, решение быстро начинает неограниченно расти, и расчёт останавливается.

Дополнительно задан порог

$$\max |p| \leq 10^6 \text{ Па.}$$

Если давление превышает этот предел, программа сообщает, на каком шаге произошла ошибка и останавливается.

2.6. Результаты

При запуске акустического расчёта импульс рождается около верхней границы головы и распространяется через неоднородную анатомическую структуру. Из-за различий в $c(\mathbf{x})$ и $\rho(\mathbf{x})$ фронт не остаётся сферическим: на границах тканей возникают отражения и изменения длины волны. Особенно заметное влияние дают костные области, где скорость звука больше, чем в мягких тканях.

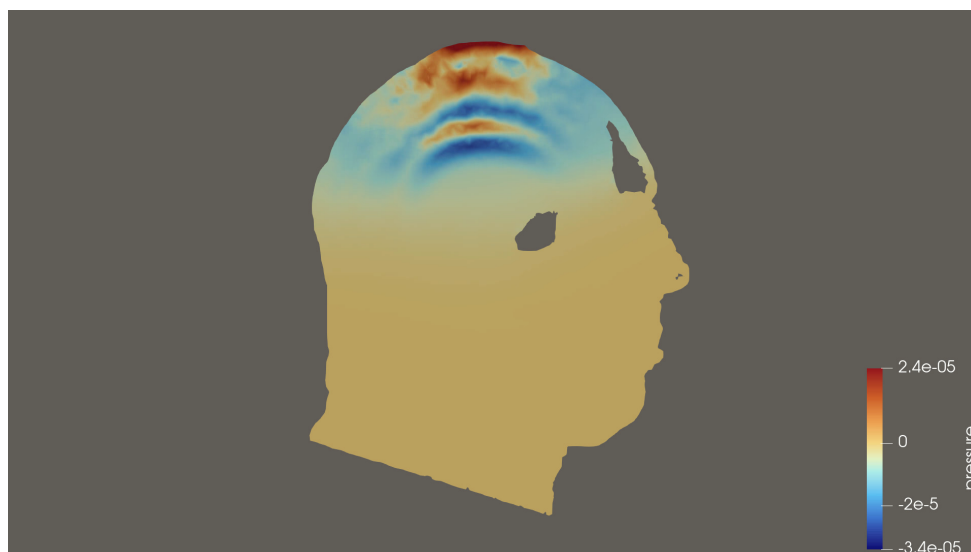


Figure 8: Кадр визуализации акустического давления в ParaView. Красно-синие области соответствуют положительным и отрицательным фазам ультразвукового импульса.

По результатам расчёта были получены VTU-кадры и видеовизуализации. Примеры видео вынесены на [Яндекс Диск](#).

2.7. Выводы

В работе собран рабочий FEM-пайплайн для моделирования ультразвукового импульса в 3D-модели головы. Главный результат — переход от готовой анатомической VTU-сетки к физической DOLFINx-модели: метки материалов сохранены, коэффициенты ρ и c назначены, а давление рассчитано явной схемой на тетраэдральной сетке.

Физическая постановка соответствует линейной акустике в неоднородной среде. Использование импульсного источника даёт короткий ультразвуковой сигнал.

III. Макропроект

1. Макропроект: тепловой блок задачи LSP

1.1. Постановка задачи

Макропроект — моделирование лазерной проковки пластины алюминия. Цель лазерной проковки состоит в том, чтобы создать в материале контролируемые пластические деформации: после разгрузки они приводят к остаточным напряжениям и изменению приповерхностного слоя. Короткий лазерный импульс нагревает поверхность металла, вызывая быстрое локальное расширение и плавление; затем возникшие термоупругие напряжения передаются в механический расчёт. Если локальное напряжение превышает предел текучести, в материале появляется пластическая зона, а после разгрузки могут остаться остаточные деформации и напряжения.

Рабочий пайплайн состоит из четырёх блоков:

1. **Тепловой код:** лазерный импульс $\rightarrow$ поле $T(\mathbf{r}, t)$ с учётом плавления.
2. **Термоупругий пересчёт:** поле температуры $\rightarrow$ линейно-упругие смещения и напряжения с температурным членом.
3. **Экспорт в VTU/VTK:** запись T , $\mathbf{u}$, компонент $\boldsymbol{\sigma}$ и σ_{vm} для передачи в механический код Вити и просмотра в ParaView.
4. **Проверка пластичности:** сравнение напряжения Мизеса с критическим напряжением материала.

1.2. Физическая постановка

1.2.1. Геометрия и масштабы

Тонкая пластина алюминия $2 \times 2 \times 0,05$ мм. Лазер падает нормально на верхнюю грань, гауссово пятно радиуса $r_L = 0,5$ мм (на уровне $1/e^2$). Боковые и нижняя грани — теплоизолированы: глубина прогрева за время импульса $\sqrt{\chi\tau} \sim 1$ мкм $\ll L_z$.

Импульс — гауссиан по времени, FWHM $\tau = 10$ нс, пиковая интенсивность $I_0 = 10^{12}$ Вт/м² — режим прямого нагрева без абляции.

1.2.2. Уравнение энергии

Энтальпийная формулировка в 3D:

$$\frac{\partial H}{\partial t} = \nabla \cdot (k(T) \nabla T) + Q(\mathbf{r}, t). \quad (34)$$

Кусочно-линейная связь $T(H)$ и интерполяция k через долю жидкой фазы переносятся из микропроекта без изменений. Опорная температура $T_{\text{ref}} = 300$ К: $H = 0$ соответствует холодному твёрдому материалу.

1.2.3. Материал

Параметр	Твёрдый Al	Жидкий Al
k , Вт/(м·К)	237	91
c , Дж/(кг·К)	900	1180
T_{melt} , К	933	
L , Дж/кг	$3,97 \cdot 10^5$	
ρ , кг/м ³	2700	

Механические константы алюминия, записываемые в `FieldData VTU`:

$$E = 70 \text{ ГПа}, \quad \nu = 0,33, \quad \alpha_T = 23,1 \cdot 10^{-6} \text{ К}^{-1}.$$

Из них в упругом расчёте вычисляются параметры Ламе:

$$\mu = \frac{E}{2(1 + \nu)}, \quad \lambda = \frac{E\nu}{(1 + \nu)(1 - 2\nu)}. \quad (35)$$

Температуропроводность $\chi_s \approx 10^{-4} \text{ м}^2/\text{с}$ — на два порядка выше, чем у льда. Это определяет на три порядка меньший шаг по времени по сравнению с микропроектом.

1.2.4. Источник лазера

Поглощённый поток на верхней грани:

$$q_{\text{abs}}(x, y, t) = (1 - R) I_0 \exp\left(-\frac{(t - t_0)^2}{2\sigma_t^2}\right) \exp\left(-\frac{2((x - x_c)^2 + (y - y_c)^2)}{r_L^2}\right). \quad (36)$$

Длина оптического проникновения в металл $\delta \sim 10 \text{ нм}$ мельче шага сетки, поэтому объёмное тепловыделение заменяется эквивалентным граничным условием Неймана на верхней грани.

1.3. Численный метод

1.3.1. Дивергентная схема

В микропроекте лапласиан аппроксимировался через $k\nabla^2 T$ с теплопроводностью в центре ячейки. На полке плавления k меняется, и такая форма нарушает локальный энергобаланс.

В макропроекте схема переписана в дивергентной форме. Поток через грань между ячейками i и $i + 1$:

$$F_{i+1/2} = -k_{i+1/2} \frac{T_{i+1} - T_i}{\Delta x}, \quad k_{i+1/2} = \frac{2 k_i k_{i+1}}{k_i + k_{i+1}}. \quad (37)$$

Гармоническое среднее — стандарт для последовательно соединённых слоёв. Поток, вышедший из ячейки, точно равен потоку, вошедшему в соседнюю — баланс энергии выполняется на уровне дискретизации.

1.3.2. Устойчивость

CFL в 3D с семиточечным шаблоном:

$$\Delta t \leq \frac{\Delta h_{\min}^2}{6 \chi_{\max}}. \quad (38)$$

При сетке $64 \times 64 \times 32$ ($\Delta z \approx 1,6$ мкм) получается $\Delta t_{\max} \sim 4 \cdot 10^{-10}$ с; с запасом 0,4 берётся $\Delta t \approx 1,7 \cdot 10^{-10}$ с. Весь импульс 200 нс — ~ 1200 шагов.

1.3.3. Граничные условия

Все грани, кроме верхней, — адиабат (зеркальное копирование энтальпии). Верхняя грань: вместо потока через несуществующую внешнюю грань подставляется $-q_{\text{abs}}(x_i, y_j, t^n)$.

1.3.4. Линейная термоупругость

После получения температурного поля каждый кадр `laser_*.vtu` читается термоупругим скриптом. Используется приближение малых деформаций:

$$\boldsymbol{\varepsilon}(\mathbf{u}) = \frac{1}{2} (\nabla \mathbf{u} + \nabla \mathbf{u}^T). \quad (39)$$

Свободное тепловое расширение задаётся членом Дюгамеля–Неймана:

$$\boldsymbol{\varepsilon}^T = \alpha_T (T - T_{\text{ref}}) \mathbf{I}. \quad (40)$$

Упругая часть деформации:

$$\boldsymbol{\varepsilon}^e = \boldsymbol{\varepsilon}(\mathbf{u}) - \boldsymbol{\varepsilon}^T. \quad (41)$$

Закон Гука для изотропного материала:

$$\boldsymbol{\sigma} = \lambda \operatorname{tr}(\boldsymbol{\varepsilon}^e) \mathbf{I} + 2\mu \boldsymbol{\varepsilon}^e. \quad (42)$$

В текущей версии решается квазистатическая задача равновесия

$$\nabla \cdot \boldsymbol{\sigma} = 0 \quad \text{в } \Omega, \quad (43)$$

с закреплением нижней грани для удаления жёстких мод. В слабой форме это эквивалентно:

$$\int_{\Omega} (\lambda \operatorname{tr} \boldsymbol{\varepsilon}(\mathbf{u}) \mathbf{I} + 2\mu \boldsymbol{\varepsilon}(\mathbf{u})) : \boldsymbol{\varepsilon}(\mathbf{v}) d\Omega = \int_{\Omega} (\lambda \operatorname{tr} \boldsymbol{\varepsilon}^T \mathbf{I} + 2\mu \boldsymbol{\varepsilon}^T) : \boldsymbol{\varepsilon}(\mathbf{v}) d\Omega. \quad (44)$$

Фазовое расширение алюминия при плавлении

$$\boldsymbol{\varepsilon}^{\text{phase}} = \frac{1}{3} \left(\frac{\Delta V}{V} \right)_{\text{melt}} f_l \mathbf{I}$$

уже записывается в исходных данных через f_l , но в итоговом расчёте оставлен только температурный член, чтобы отделить термоупругий вклад от вклада фазового перехода.

1.3.5. Критерий пластичности

Для оценки, останется ли расчёт в упругой области, используется эквивалентное напряжение Мизеса:

$$\sigma_{vm} = \sqrt{\frac{3}{2} \mathbf{s} : \mathbf{s}}, \quad \mathbf{s} = \boldsymbol{\sigma} - \frac{1}{3} \text{tr}(\boldsymbol{\sigma}) \mathbf{I}. \quad (45)$$

Критерий текучести:

$$\sigma_{vm} \geq \sigma_y. \quad (46)$$

В постпроцессоре записываются:

$$R_y = \frac{\sigma_{vm}}{\sigma_y}, \quad \sigma_{over} = \max(\sigma_{vm} - \sigma_y, 0), \quad (47)$$

а также бинарные поля `plastic_zone` и `plastic_zone_ever`. Если $R_y < 1$, деформация в данной точке остаётся упругой; если $R_y \geq 1$, линейно-упругая модель предсказывает достижение текучести, и в реальном материале следует ожидать пластическую разгрузку и остаточную деформацию.

1.4. Реализация

1.4.1. Архитектура

Raylib и сцены из микропроекта удалены. Тепловой блок оставлен как headless C++-солвер:

- `core/` — `Grid`, `Solver`, `SimMaterial`. Headless, без внешних зависимостей.
- `scenes/` — единственная сцена `LaserPulse`: однородный старт $T = 300$ K, q_{abs} на верхней грани.
- `export/` — `VtkWriter` (VTU + временная серия .pvd), `ProbeWriter` (CSV-зонд в центре пятна).

Механическая часть вынесена в отдельные Python-скрипты на DOLFINx:

- `thermal_stress_from_vtu.py` — код термоупругости. Он читает последовательность `laser_*.vtu`, строит тетраэдральную сетку DOLFINx из VTK-файла, переносит узловое поле температуры T в пространство P_1 , читает из `FieldData` константы E , ν , α_T и T_{ref} . Для каждого кадра решается статическая термоупругая задача с закреплением нижней грани; матрица жёсткости собирается один раз, а правая часть пересобирается при изменении температуры.
- `elastic_waves.py` — код линейной упругости для распространения упругих волн. В нём используется тот же тетраэдральный VTU-формат, векторное пространство P_1 для смещений, закон Гука $\boldsymbol{\sigma} = \lambda \text{tr}(\boldsymbol{\epsilon}) \mathbf{I} + 2\mu \boldsymbol{\epsilon}$ и схема Ньюмарка по времени. Лазерное воздействие в этом модуле задаётся как гауссово давление на отмеченной поверхностной области.
- `postprocess_von_mises.py` — постпроцессор, который читает `von_mises` из термоупругих VTU-файлов и добавляет поля `yield_ratio`, `overstress`, `plastic_zone` и `plastic_zone_ever`.

Такое разделение оставляет тепловой код независимым от DOLFINx, а механические расчёты получают данные только через VTU/PVD-интерфейс. Сборка теплового блока — один вызов CMake без `find_package`; механические скрипты запускаются отдельно в окружении DOLFINx.

1.4.2. Экспорт

Декартова сетка переупаковывается в неструктурированную тетраэдрическую: каждая ячейка бьётся на 5 тетраэдров без вспомогательных узлов (тип VTK_TETRA). В каждом кадре:

- **PointData:** T , H , liquid_fraction, phase;
- **FieldData:** 13 материальных констант (теплофизика, оптика, механика) — напарник читает их из того же файла.

Один кадр (131 072 узла, 615 195 тетраэдра) — ≈ 17 МБ.

Термоупругий пересчёт создаёт вторую временную серию thermal_stress_*.vtu. В ней дополнительно записаны:

- displacement, displacement_magnitude;
- stress_trace, von_mises;
- sigma_xx, sigma_yy, sigma_zz, sigma_xy, sigma_xz, sigma_yz.

1.5. Результаты

Результаты моделирования в видеоформате расположены на [Яндекс Диске](#).

При $I_0 = 10^{12}$ Вт/м², $\tau = 10$ нс, $R = 0,3$ температура в центре пятна поднимается с 300 К до ≈ 1000 К за ~ 30 нс, после чего выходит на полку $T_{\text{melt}} = 933$ К: дальнейшая энергия уходит в теплообмен с окружающей средой. Зона $f_l > 0$ — приповерхностный слой глубиной несколько микрон.

Грубая энергетическая оценка даёт глубину расплава ~ 1 мкм — согласуется с расчётом.

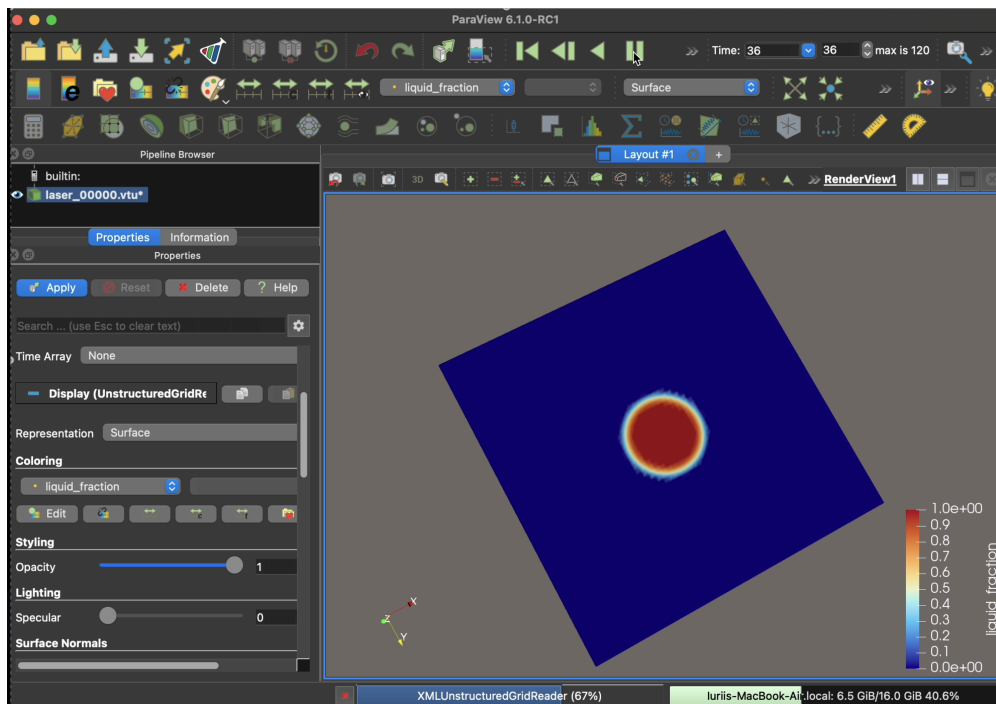


Figure 9: Поле доли жидкой фазы liquid_fraction в ParaView. Расплав возникает в центре лазерного пятна и остаётся приповерхностным.

Для термоупругого этапа было обработано 601 поле температуры на интервале от 0 до 1,0 мкс. Постпроцессинг пластичности выполнен для предела текучести $\sigma_y = 250$ МПа. Первое превышение критерия Мизеса появилось на $t \approx 26,7$ нс:

$$\max \sigma_{vm} \approx 3,84 \cdot 10^8 \text{ Па}, \quad R_y^{\max} \approx 1,54.$$

В этот момент в пластическую зону попало 172 узла из 131072, то есть около 0,13% сетки.

Максимальное напряжение за весь прогон достигнуто около $t \approx 43,4$ нс:

$$\max \sigma_{vm} \approx 1,57 \cdot 10^9 \text{ Па}, \quad R_y^{\max} \approx 6,29.$$

Наибольшее число узлов, когда-либо попавших в пластическую зону, составило 2018 из 131072 (1,54%) около $t \approx 192$ нс. К концу расчёта, при $t \approx 1,0$ мкс, максимальное напряжение всё ещё выше выбранного предела текучести:

$$\max \sigma_{vm} \approx 4,29 \cdot 10^8 \text{ Па}, \quad R_y^{\max} \approx 1,72.$$

Таким образом, при выбранных параметрах нагрева расчёт не остаётся целиком в упругой области. Основной объём пластины работает упруго, но в окрестности лазерного пятна линейно-упругая оценка превышает критическое напряжение, поэтому там ожидается пластическая релаксация и остаточная деформация.

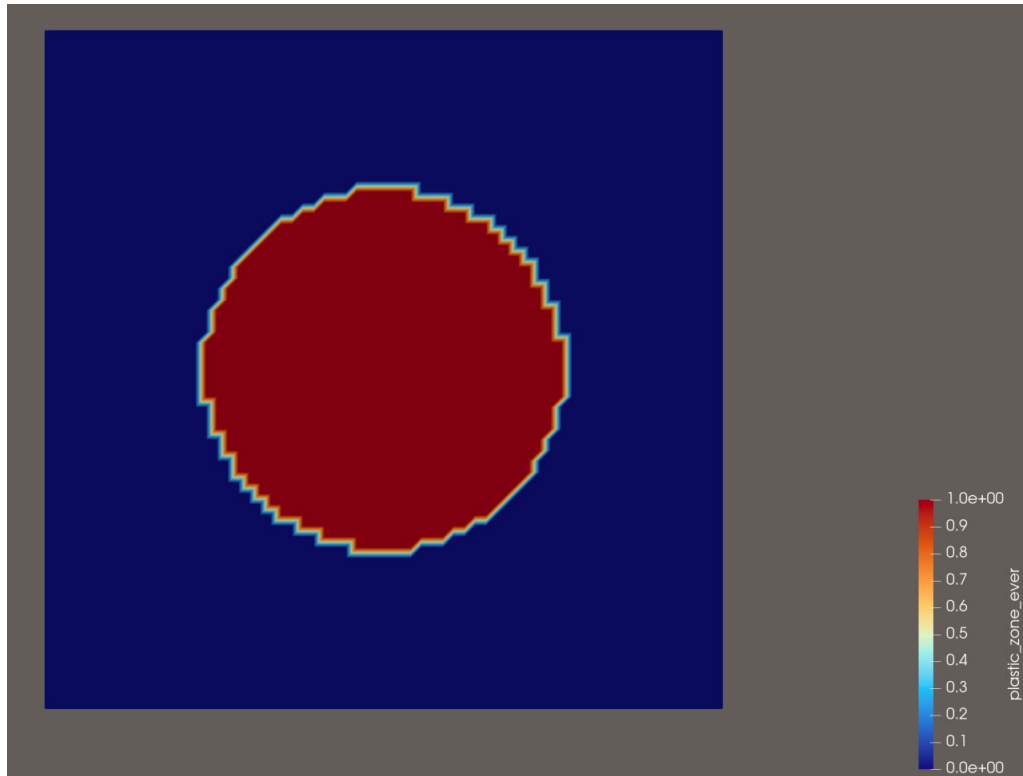


Figure 10: Накопленная зона пластичности `plastic_zone_ever`, вид сверху. Красным цветом отмечены узлы, в которых за время расчёта хотя бы один раз выполнялось условие $\sigma_{vm} \geq \sigma_y$. Область повторяет форму лазерного пятна.

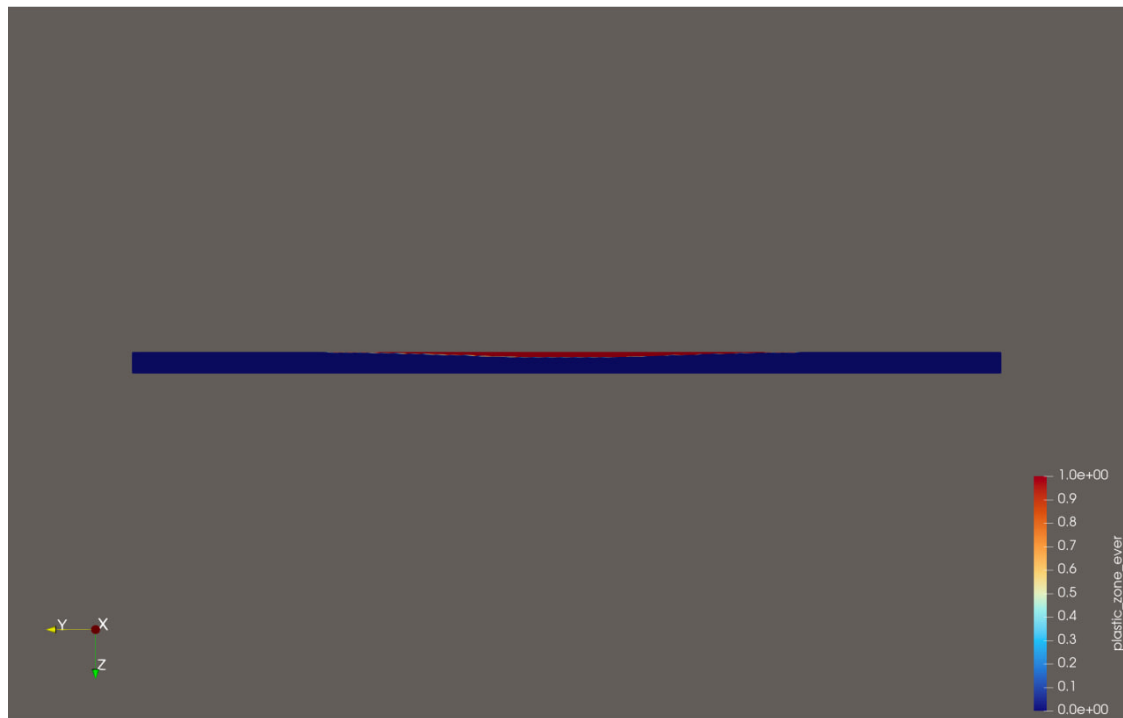


Figure 11: Накопленная зона пластичности `plastic_zone_ever`, сечение по толщине пластины. Красный слой показывает, что неупругая деформация локализована в тонкой приповерхностной области около зоны лазерного нагрева.

1.6. Направления развития

1. **Фазовый вклад в деформацию.** Добавить в термоупругий расчёт член ϵ^{phase} , зависящий от `liquid_fraction`; это даст вклад объёмного скачка при плавлении.
2. **Абляция (формула Fabbro).** При $I_0 > 10^{13}$ Вт/м² доминирует плазменное давление; подаётся в механический код отдельно как Neumann-BC на поверхности.
3. **Произвольная геометрия.** Сейчас — параллелепипед. Минимальная переделка — маска «внутри/снаружи» поверх декартовой сетки.

1.7. Выводы

Реализован 3D тепловой блок для задачи LSP на основе энтальпийного метода и доведён следующий этап: линейно-упругий расчёт напряжений по температурному полю. Ключевые изменения относительно 2D-микропроекта: дивергентная форма потоков с гармоническим усреднением k (точный энергобаланс), замена среды на алюминий, лазерный источник как Neumann-ГУ, экспорт в тетраэдрический VTU с временной серией.

Физическая картина — глубина и размер зоны плавления — согласуется с энергетическими оценками. Термоупругий расчёт показывает локальное превышение предела текучести в зоне лазерного пятна, поэтому для выбранного режима ожидается не только упругая деформация, но и малая приповерхностная область остаточной пластичности. Тем самым цель лазерной проковки в расчёте достигнута: выбранный импульс создаёт локальную зону пластической деформации в материале.

References

- [1] Абрамовиц М., Стиган И. *Справочник по специальным функциям*. Наука, 1979.
- [2] Schroeder W., Martin K., Lorensen B. *The Visualization Toolkit*. Kitware, 4th ed., 2006.
- [3] Logg A., Mardal K.-A., Wells G. *Automated Solution of Differential Equations by the Finite Element Method*. Springer, 2012.